

ATI Perception Challenge – Rim Center Identification Design Plan Submission

Author: Amr Attia

Date: September 11, 2025

1. Problem Understanding

The goal is to design a system that automatically identifies the center of car wheels from close-up camera images, even when the wheels are tilted, rotated, partially occluded, or absent.

- **Input:** A single RGB image from a 15 FPS stream
- **Output:**
 - Pixel coordinates of the wheel center if a wheel is visible
 - “No wheel” if no wheel is present in the frame

The ground truth “center” is defined as the center of the ellipse formed by projecting the inner edge of the tire onto the image plane.

2. Approach Overview

I propose a **heatmap-based regression approach** instead of direct coordinate regression. The system will generate a probability heatmap where the peak corresponds to the wheel center.

Advantages:

- Encodes spatial uncertainty
- Improves training stability
- Enables thresholding for “no wheel” detection

Pipeline Steps:

1. **Preprocessing**
 - Resize to 256×256
 - Normalize intensities
 - Augment with rotation, brightness, contrast, and flips
2. **Model Architecture**
 - Compact UNet-like encoder–decoder
 - Input: RGB image
 - Output: single channel heatmap
 - Activation: Sigmoid
3. **Training Objective**
 - Ground truth: Gaussian blob centered at wheel center ($\sigma = 3\text{--}5$ px)
 - Loss: MSE for heatmap + BCE for wheel presence
4. **Inference**
 - Argmax of heatmap $\rightarrow (x, y)$
 - If max probability $<$ threshold $\tau \rightarrow$ classify as “no wheel”

5. Post-Processing

- Optional subpixel refinement via quadratic fitting
- Return coordinate and confidence score

3. Justification for Approach

- Heatmap regression is state-of-the-art in keypoint detection
- Handles ambiguity and partial visibility
- Lightweight and real-time capable
- Easily extensible for multi-wheel detection

4. Data Considerations

- **Provided Dataset:** 1,000 labeled examples
- **Additional Needs:**
 - More diversity in car models, lighting, and motion blur
 - Negative samples (no wheels)
 - Augmentations: $\pm 15^\circ$ rotation, blur, scale jitter

5. Testing & Validation Strategy

Evaluation Metrics:

Metric	Description
Mean Pixel Error	Average Euclidean distance to ground truth
Median Pixel Error	Robust to outliers
% within 5px / 10px	Accuracy thresholds
False Positive Rate	Misclassification on “no wheel” frames
Inference Latency	Time per frame on target hardware

Validation Strategy:

- 80/20 train/Val split
- Cross-validation if needed
- Visual overlay inspection
- Stress tests: low light, blur, occlusion

6. Edge Case Handling

Scenario	Strategy
No wheel present	Threshold on max heatmap confidence
Multiple wheels	Return strongest peak (extendable to multi-peak)
Partial occlusion	Heatmap diffuses, argmax remains plausible
Extreme tilt	Handled via labeled training data
Decorative rims	Robustness via diverse training samples

7. Sensor & Environmental Notes

- **Sensor:** RGB camera
- **Frame rate:** 15 FPS
- **Lighting:** IR/depth optional for future robustness
- **Mounting:** Ensure wheel radius > 30 px in resized image

8. Implementation Timeline

Phase	Tasks	Time Estimate
Prototype	Dataset loader, baseline UNet, training loop	2–3 days
Refinement	Augmentations, classifier, subpixel tuning	1 week
Deployment	ONNX/TorchScript, integration, embedded testing	2–3 weeks

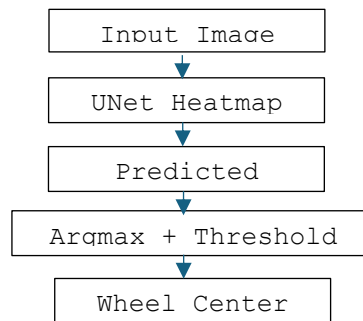
Risk Level: Low to medium. Most risk lies in dataset variability and false positives.

9. Future Extensions

- Temporal smoothing (e.g., Kalman filter)
- Depth/LiDAR fusion
- Object detection pre-filter for multi-wheel scenes
- Synthetic data via domain randomization

10. Visual Pipeline Diagram

Code



11. Conclusion

The proposed heatmap-based UNet system offers a robust, efficient, and extensible solution to rim center identification. It balances precision with simplicity and is well-suited for real-world deployment.

12. Extra Work Beyond Requirements

- GT Relabeling: Added Relabeling pipeline for annotation drift_1.0.py to fix inaccurate/missing centers (e.g., rim_974).
- Synthetic Data: Built Synthetic Data Generator_with_Heatmap_Output.ipynb to expand and balance the dataset with augmented images + heatmaps.

13. Next Steps

- Enhance current heatmap: Increase resolution, add sub-pixel offsets, fuse multi-scale outputs, and include uncertainty maps.
- Hybrid methods: Combine heatmap with coordinate head or segmentation mask; apply geometry priors to reduce false positives.
- Transformer models: Test ViT/Swin/DETR-style keypoint detection for better context handling.
- Self-supervised pretraining: Use MAE/SimCLR to improve generalization; fine-tune with minimal labels.
- Temporal/multi-view fusion: Apply Kalman filtering or attention to stabilize predictions; fuse multi-camera views if available.
- Long-term R&D: Develop multi-task, uncertainty-aware, transformer-based architecture with embedded motion/shape constraints.